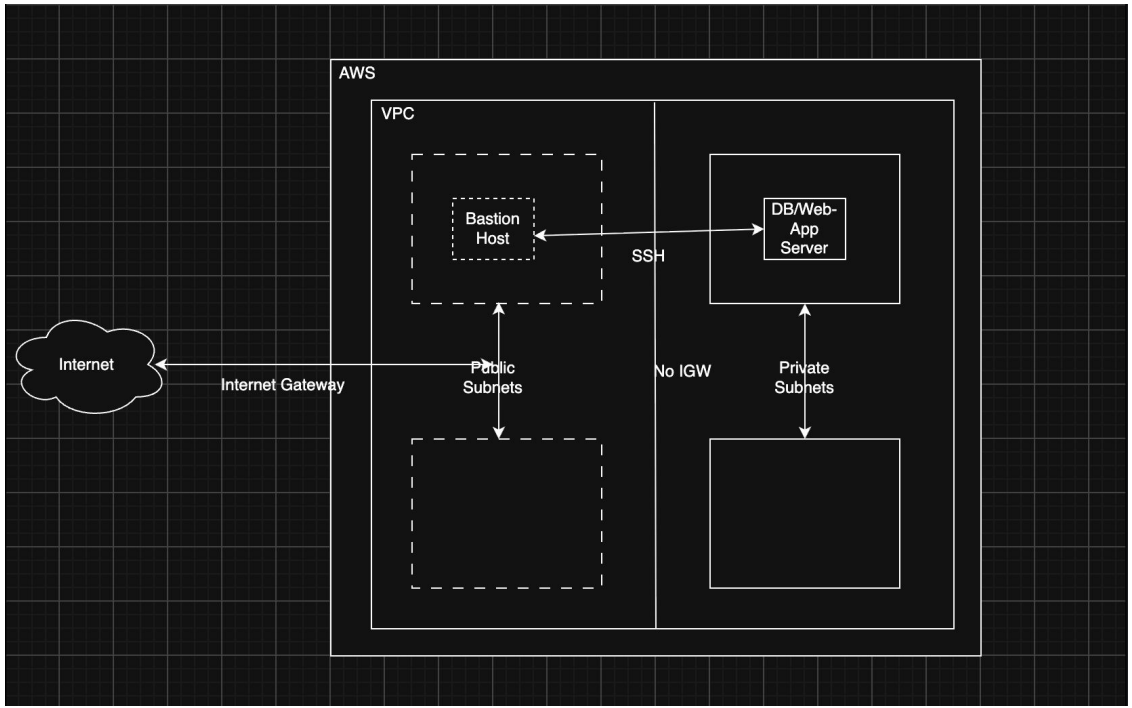
Cloud Platform Deep Dive M2:W3



Databases, Serverless & Application Services

Every app you use, from your bank to Netflix, is a front end talking to a database. Today we store data and run code without managing a single server.

Last week, we talked about private subnets and how they are where databases live.



SQL vs NoSQL

SQL (relational): Data in tables with rows, columns, and a fixed schema. Tables link through keys, and you query with SQL (Structured Query Language). Strong consistency. Examples: MySQL, PostgreSQL.

Analogy: an Excel workbook with strict columns and linked sheets.

NoSQL: Flexible structures (key-value, document, graph, column). No fixed schema, scales horizontally, very fast for simple lookups. Examples: DynamoDB, MongoDB, Firestore.

Analogy: a filing cabinet where each folder can contain different things.

In essence, no DB is straight up “better” or “faster.” They are different, with trade-offs.

Self-managed vs managed Databases

Self-managed means installing MySQL on a VM: you handle patching, backups, failover, and scaling while Managed (RDS / Cloud SQL) means the provider handles all that; you handle schema, queries, and access.

What managed DB gives you: automated backups, point-in-time recovery, patching, Multi-AZ failover (a standby in another AZ), read replicas, monitoring.

What you still own: data design, security groups, credentials, cost.

Example engines on RDS: MySQL, PostgreSQL, MariaDB, SQL Server, Oracle, and Aurora (AWS cloud-optimised version).

Creating a Managed MySQL DB on AWS RDS(Lab)

Cost warning: RDS bills while running even when idle. Delete it after the demo, or stop it (AWS restarts stopped instances after 7 days).

Part 1: Create the app VM (the “client”)

1. Go to EC2 ..> Launch instance.
2. Name: db-client, AMI: Ubuntu 22.04 or 24.04, type: t2.micro or t3.micro.
3. Key pair: use existing one.
4. Network: default VPC, allow SSH from your IP only.
5. Launch, then note the instance’s security group name (e.g. launch-wizard-1). You’ll reference it in step 9.

Part 2: Create the database

6. Go to RDS → Create database.
7. Choose Full Configuration, engine MySQL.
8. Template: Free tier (Sandbox).
9. Settings:
    - DB instance identifier: `lab-db`
    - Master username: `admin`
    - Credentials: set a password (save it somewhere safe; never commit it to Git)
10. Instance configuration: `db.t3.micro` (or the free-tier option shown).
11. Storage: 20 GB, and disable storage autoscaling to avoid surprises.
12. Connectivity: Don’t connect to an EC2 compute resource, “Connect to an EC2 compute resource” would automatically create the security group rules between your VM and the database instead of letting you add the rule manually in Part 3, this is where you set the VM’s security group as the source.
    - VPC: default
    - Public access: No
    - VPC security group: Create new, name it `rds-lab-sg`
13. Additional configuration: set Initial database name to **`labdb`**. Turn off automated backups and Enhanced Monitoring for the lab (saves cost and time).
14. Click Create database. Status will show Creating, which takes 5-10 minutes.

Part 3: Lock down access (least privilege)

15. While it builds, go to **EC2 → Security Groups → rds-lab-sg → Edit inbound rules**.
16. Remove any wide-open rules and add:
    - Type: **MySQL/Aurora (3306)**
    - Source: **the db-client VM's security group** (select the SG, don't type an IP)
17. Save. This means only your VM can reach the database..

Part 4: Connect and run SQL

18. When status is **Available**, open the database and copy the **Endpoint** (e.g. lab-db.xxxx.region.rds.amazonaws.com).
19. SSH into db-client and install the client:

```
sudo apt update && sudo apt install mysql-client -y
```

20. Connect using: mysql -h <endpoint> -u admin -p
21. Enter your password. You should see the mysql> prompt.

```
SHOW DATABASES;
USE labdb;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50),
  course VARCHAR(50)
);

INSERT INTO students (name, course)
VALUES ('Your Name', 'Cloud Engineering');

SELECT * FROM students;
```

23. Screenshot the output, then type `exit`.

Part 5: Prove it's private

24. From **your own laptop**, try the same `mysql -h <endpoint> -u admin -p` command. It should **time out**, because the DB has no public access and the SG only trusts your VM. Good practice

Part 6: Explore what "managed" means

26. In the RDS console, find: the **Configuration** tab, **Maintenance & backups**, and the **Monitoring** graphs. Note one thing AWS is handling that you would have to do yourself on a self-managed VM.

Part 7: Cleanup (required)

27. RDS → select `lab-db` → **Actions** → **Delete**. Untick "create final snapshot" and "retain automated backups," then confirm.
28. Terminate the `db-client` VM.
29. Delete `rds-lab-sg` once the DB is gone.
30. Check the RDS and EC2 consoles are empty.

Common problems (for you to watch)

- Connection timed out from the VM: SG source is wrong, or the DB and VM are in different VPCs. Check the SG rule first.
- Access denied for user: wrong password or username.
- `mysql: command not found`: the client package wasn't installed.
- Create button blocked: no default VPC in the region, or a missing DB subnet group (the default VPC normally handles this).
- "Free tier" option missing: the account is no longer on the legacy free tier, in which case use the smallest `db.t3.micro` and delete it immediately.
- Forgot cleanup: RDS keeps billing, which is the main cost risk.

NoSQL with DynamoDB (Lab)

Part 1: Create the table

1. Go to DynamoDB → Tables → Create table.
2. Table name: `students`
3. Partition key: `id` (type String)
4. Leave the sort key empty.
5. Table settings: Customize settings, capacity mode On-demand.
6. Leave encryption at the default, then click Create table. It becomes Active in under a minute.

Part 2: Add items (see the flexible schema)

7. Open `students` → Explore table items → Create item.
8. Add the first item (switch to JSON view for speed):

```
{
  "id": "1",
  "name": "Ada",
  "course": "Cloud Engineering",
  "score": 85
}
```

```
{
  "id": "2",
  "name": "Tunde",
  "course": "DevOps",
  "city": "Lagos",
  "skills": ["Linux", "Git", "Bash"]
}
```

9. Create a second item with **different attributes**:
10. Create a third item with only `id` and `name`.

Part 3: Read data three ways

12. **Scan** (reads everything): in *Explore table items*, click **Scan** and run it. Note that it returns all items.
13. **Query** (reads by key, efficient): switch to **Query**, enter partition key `id = 2`, run it. Only Tunde returns.
14. **Filter:** in Scan, add a filter `course = DevOps` and run it.**Part 4: Update and delete**
16. Edit item 1: change `score` to `90` and add a new attribute `passed` = `true`. Save.
17. Delete item 3.

Part 5: Use the CLI (optional but valuable)

18. Open **CloudShell** (the terminal icon in the console top bar). It's already authenticated.
19. Add an item:

```
aws dynamodb put-item \
  --table-name students \
  --item '{"id":{"S":"4"},"name":{"S":"Chidi"},"course":{"S":"Security"}}'
```

```
aws dynamodb get-item \
  --table-name students \
  --key '{"id":{"S":"4"}}'
```

21. Scan the table: aws dynamodb scan --table-name students

Part 7: Cleanup

26. Delete the table: select `students` → Delete, untick “create backup,” confirm.
27. Check the DynamoDB tables list is empty.

Common problems

- Wrong data type: the key was created as Number but the item uses a string like `"1"`, which gives a type mismatch error. Match the type exactly.
- Table “Creating” for a while: wait for Active before adding items.
- Typed JSON confusion in the CLI: missing the `{"S": ...}` wrapper causes validation errors.
- Using Scan out of habit: reinforce Query for lookups.

Serverless and Lambda

Serverless means Servers still exist, but you don’t provision, patch, or scale them. You upload code and pay only when it runs. Lambda on AWS provides this service.

Lambda basics:

- Function: your code (Python, Node.js, etc.)
- Trigger: what starts it (HTTP request, S3 upload, schedule, queue message)
- Execution role: IAM role that controls what the function can access (least privilege again)
- Limits: max 15 minutes per run, memory/timeouts you configure
- Pricing: per request plus compute time, with a generous free tier

Your First Lambda Function (Lab)

Goal: Create a Lambda in the console, paste simple code, test it with the Test button, and show the CloudWatch logs.

1. Create the function

1. Go to **Lambda** → **Create function** → **Author from scratch**.
2. Name: `hello-lambda`, Runtime: **Python 3.x**, leave the default execution role (it auto-creates basic logging permissions).
3. Click **Create function**.

2. Add the code

4. In the **Code** tab, replace the contents with:

```
import json

def lambda_handler(event, context):
    name = event.get("name", "World")
    print(f"Function invoked with name: {name}")
    return {
        "statusCode": 200,
        "body": json.dumps(f"Hello, {name}!")
    }
```

5. Click Deploy (don’t skip this, as it’s the most common mistake).

3. Test it

6. Click Test → Create new test event, name it `test1`, and use:

json

```
{ "name": "Ada" }
```

7. Click Test. Check the Execution results: response should be `Hello, Ada!` with status 200.
8. Run it again with `{}` to see the default `Hello, World!`.

4. View the logs

9. Go to the Monitor tab → View CloudWatch logs.
10. Open the latest log stream and find your `print` line, plus the `START`, `END`, and `REPORT` lines (note Duration and Billed Duration).

5. Break it on purpose (1 min)

11. Change `event.get("name")` to `event["name"]`, Deploy, and test with `{}`. Observe the `KeyError` in the results and in CloudWatch. This shows students how to read errors.

6. Cleanup

12. Delete the function (Actions → Delete), then delete its log group in CloudWatch and its auto-created IAM role.

Take note of the `REPORT` line: That Billed Duration is what you pay for. When the function isn’t running, you pay nothing.

Application Services

Application services are managed cloud services that help you build, run, and connect applications, so you don't have to build those pieces yourself.

The main categories

App hosting (run your code without managing servers): You upload code or a container, and the platform handles servers, scaling, and patching.

- AWS: Elastic Beanstalk, App Runner, ECS Fargate, Amplify
- GCP: App Engine, Cloud Run

API management (expose your app to the world): Creates the HTTP endpoints, handles authentication, throttling, and routing to Lambda or your backend.

- AWS: API Gateway, GCP: API Gateway / Apigee

Messaging and queues (connect parts of an app): Lets components talk without being directly connected. Example: an order is placed, a message goes to a queue, and a separate service processes it later. If one part fails, messages wait instead of being lost.

- AWS: SQS (queue), SNS (notifications), EventBridge (events)
- GCP: Pub/Sub

Caching (make apps faster): Stores frequently read data in memory so the database isn't hit every time.

- AWS: ElastiCache, GCP: Memorystore

Authentication and identity for app users: Handles sign-up, login, and passwords for your app's users (different from IAM, which is for your cloud engineers).

- AWS: Cognito, GCP: Identity Platform / Firebase Auth

Application Services (Contd.)

Content delivery: Serves static content (images, pages) from locations near users for speed.

- AWS: CloudFront, GCP: Cloud CDN

Workflow and automation: Chains functions and steps into a process.

- AWS: Step Functions, GCP: Workflows

Moving down the table, you manage less and the provider manages more.

Service	**What it does**	**Control**
EC2 / Compute Engine	Your own VM	Most
Elastic Beanstalk / App Engine	Upload code, platform runs it	Medium
ECS Fargate / Cloud Run	Run containers without servers	medium-Low
Lambda / Cloud Functions	Run functions on events	Least
API Gateway	Expose HTTP endpoints in front of functions	Supporting
Amplify	Host front-end apps	Supporting

Quiz

1. Name one use case each for SQL and NoSQL.
2. What does a managed database handle for you?
3. Why does the RDS instance go in a private subnet?
4. What triggers a Lambda function? Give two examples.
5. What is a cold start?

Extra Learning Materials…

Extra learning materials

SQL basics (interactive, good for beginners)

- SQLBolt, short interactive lessons with exercises: https://sqlbolt.com/
- W3Schools SQL tutorial, with a “Try it” editor: https://www.w3schools.com/sql/
- SQLZoo, practice queries with instant feedback: https://sqlzoo.net/
- Mode SQL Tutorial, beginner to advanced: https://mode.com/sql-tutorial

MySQL (the engine used in the lab)

- Official MySQL reference manual: https://dev.mysql.com/doc/refman/8.4/en/
- MySQL tutorial section (connecting, creating tables, queries): https://dev.mysql.com/doc/refman/8.4/en/tutorial.html

AWS database services

- Amazon RDS user guide: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html
- Amazon DynamoDB developer guide: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html
- Choosing between SQL and NoSQL on AWS: search “AWS What is NoSQL” on https://aws.amazon.com/nosql/

Serverless

- AWS Lambda developer guide: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
- Serverless on AWS overview: https://aws.amazon.com/serverless/

Google Cloud equivalents (if students want to compare)

- Cloud SQL docs: https://cloud.google.com/sql/docs
- Firestore docs: https://cloud.google.com/firestore/docs
- Cloud Functions docs: https://cloud.google.com/functions/docs